Espen Oland, 2025
Tom Stian Andersen, 2026

# Assignment 2: Orbital Mechanics

## 1 Introduction

In this assignment the student will learn how to simulate orbital mechanics using classical and alternative orbital parameters. This is mainly done by extending 'orbit_lib.py' with useful functions that simplify the implementation of scenarios that simulate orbital mechanics.

After completing this assignment the student will learn:

- How to implement orbital mechanics and propagate the motion of satellites.

# 2 Preliminaries

Before describing the steps needed to complete todays assignment, some fundamentals are needed. Specifically, the fundamentals of orbital mechanics. This section is is based on Lecture 3.

## 2.1 Kepler's laws of planetary motion

1. the orbit of each planet is an ellipse with the Sun occupying one focus;
2. the line joining the Sun to a planet sweeps out equal areas in equal intervals of time;
3. a planet's orbital period is proportional to the mean distance between the Sun and the plane, raised to the power 3/2.

## 2.2 Earth Model

We assume the following about the Earth

- Spherical with radius $R_E = 6378.1363$ km
- Constant rotational velocity $\omega_E = 7.292115 \cdot 10^{-5}$ rad/s
- Uniform density $\mu = 398604.415$ km$^3$/s$^2$

## 2.3 Rotations

Rotation matrices representing abritrary orientations can be constructed using Euler angles through combining the basic rotation matrices in various sequences

$$\boldsymbol{R}_1(\alpha) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\alpha & -\sin\alpha \\ 0 & \sin\alpha & \cos\alpha \end{bmatrix}$$

$$\boldsymbol{R}_2(\beta) = \begin{bmatrix} \cos\beta & 0 & \sin\beta \\ 0 & 1 & 0 \\ -\sin\beta & 0 & \cos\beta \end{bmatrix}$$

$$\boldsymbol{R}_3(\gamma) = \begin{bmatrix} \cos\gamma & -\sin\gamma & 0 \\ \sin\gamma & \cos\gamma & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Two common sequences to construct rotation matrices is the Roll $\phi$, pitch $\theta$, yaw $\psi$ sequence

$$\boldsymbol{R} = \boldsymbol{R}_3(\psi)\,\boldsymbol{R}_2(\theta)\,\boldsymbol{R}_1(\phi)$$

and the Classical Euler sequence

$$\boldsymbol{R} = \boldsymbol{R}_3(\Omega)\,\boldsymbol{R}_1(i)\,\boldsymbol{R}_3(\omega)$$

This can also be done with quaternions

$$\boldsymbol{q}_1(\alpha) = \begin{bmatrix} \cos\frac{\alpha}{2} \\ \sin\frac{\alpha}{2} \\ 0 \\ 0 \end{bmatrix}, \qquad \boldsymbol{q}_1(\beta) = \begin{bmatrix} \cos\frac{\beta}{2} \\ 0 \\ \sin\frac{\beta}{2} \\ 0 \end{bmatrix}, \qquad \boldsymbol{q}_1(\gamma) = \begin{bmatrix} \cos\frac{\gamma}{2} \\ 0 \\ 0 \\ \sin\frac{\gamma}{2} \end{bmatrix}$$

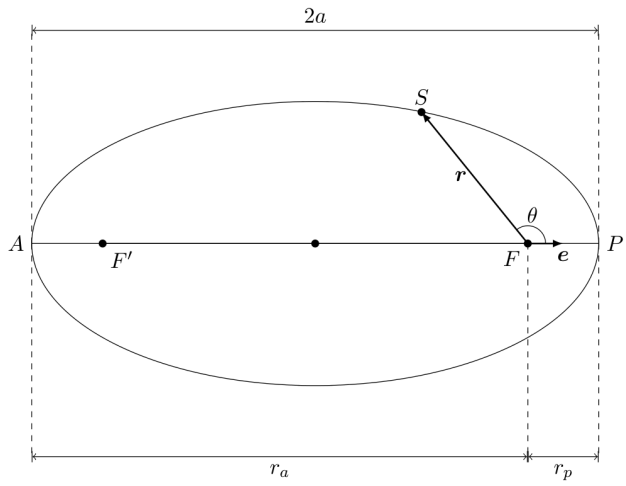


Figure 1: Position of the Satellite $S$ is given by $(r, \theta)$.

with the roll, pitch, yaw sequence being

$$\boldsymbol{q} = \boldsymbol{q}_3(\psi) \otimes \boldsymbol{q}_2(\theta) \otimes \boldsymbol{q}_1(\phi)$$

and the Classical Euler sequence

$$\boldsymbol{q} = \boldsymbol{q}_3(\Omega) \otimes \boldsymbol{q}_1(i) \otimes \boldsymbol{q}_3(\omega).$$

## 2.4 Classical Orbital parameters

To be able to visualise the orbits of satellites we need to be able to compute their position with respect to the ECI frame. This can be done using the classical orbital parameters

$h$ - Specific relative angular momentum

$e$ - Eccentricity

$\theta$ - True anomaly

$\Omega$ - Right ascension of the ascending node (RAAN)

$i$ - Inclination

$\omega$ - Argument of perigee

where $h$, $e$ and $\theta$ tell us the position of the satellite in its orbital plane (see Figure 1) and $\Omega$, $i$ and $\omega$ tells us the orientation of the orbital plane with respect to the ECI frame (see Figure 2).

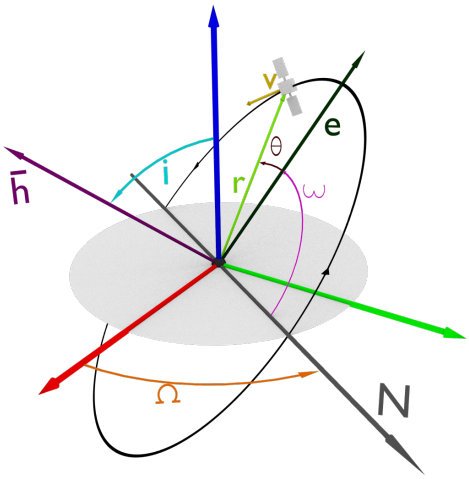


Figure 2: The orientation of the orbital plane with respect to the ECI frame, defined by the angles $\Omega$, $i$ and $\omega$.

## 2.5 Alternative Orbital parameters

A limitation of the Classical orbital parameters is that they do not include any notion of time. The alternative orbital parameters include time through the mean motion and mean anomaly

$n$ - Mean motion

$e$ - Eccentricity

$M_e$ - Mean anomaly

$\Omega$ - Right ascension of the ascending node (RAAN)

$i$ - Inclination

$\omega$ - Argument of perigee

where $n$, $e$, $M_e$ tells us the position of the satellite in its orbital plane. We will briefly outline the relation between $n$, $M_e$ and $t$. The rate of change of true anomaly is related to $h$ through

$$\frac{d\theta}{dt} = \frac{h}{r^2}$$

which by inserting the orbital equation for $r$ and rearranging we get the form

$$\frac{\mu}{h^3} dt = \frac{d\theta}{1 + e\cos\theta}$$

Integrating on both sides gives

$$\int_{t_p}^{t} \frac{\mu}{h^3}\, \mathrm{d}\tau = \int_0^{\theta} \frac{1}{1 + e\cos\vartheta}\, \mathrm{d}\vartheta$$

where $t_p$ was the time at the last periapsis crossing (i.e. last time $\theta = 0$). If we assume we are in a elliptical orbit ($e < 1$) then the solution to the integral is

$$\left(1-e^2\right)^{\frac{3}{2}} \frac{\mu^2}{h^3} t = 2 \arctan\left(\sqrt{\frac{1-e}{1+e}} \tan\frac{\theta}{2}\right) - e\frac{\sqrt{1-e^2}\sin\theta}{1+e\cos\theta} \tag{1}$$

where we have assumed that $t_p = 0$. If we define

$$\begin{aligned} M_e &= \left(1-e^2\right)^{\frac{3}{2}} \frac{\mu^2}{h^3} t = \frac{2\pi}{T} t = nt \\ \sin E &= \frac{\sqrt{1-e^2}\sin\theta}{1+e\cos\theta} \\ E &= 2\arctan\left(\sqrt{\frac{1-e}{1+e}} \tan\frac{\theta}{2}\right) \end{aligned}$$

then (1) takes the form

$$M_e = E - e\sin E$$

which is called Kepler's equation. The mean anomaly is related to the mean motion through

$$M_e = nt = \frac{2\pi}{T} t$$

It is important to note that $t$ in this expression refers to the time since the last periapsis crossing. Therefore $t$ lies in the range $[0, T]$ while $M_e$ lies in the range $[0, 2\pi]$.

## 2.6 Classical to Alternative

We can obtain the alternative orbital parameters from the classical ones. From $\theta$ we compute $E$

$$E = 2\arctan\left(\sqrt{\frac{1-e}{1+e}} \tan\frac{\theta}{2}\right)$$

then use Kepler's Equation to find $M_e$

$$M_e = E - e\sin E$$

The semi-major axis can be computed from $h$ and $e$

$$a = \frac{h^2}{\mu}\left(\frac{1}{1-e^2}\right)$$

which allows us to compute the orbital period

$$T = \frac{2\pi}{\sqrt{\mu}} a^{\frac{3}{2}}$$

and consequently the mean motion

$$n = \frac{2\pi}{T}$$

## 2.7 Alternative to Classical

We can obtain the classical orbital parameters from the alternative ones. From the mean motion we obtain the orbital period

$$T = \frac{2\pi}{n}$$

which allows us to compute $a$ and therefore $h$

$$a = \left(\frac{\sqrt{\mu}T}{2\pi}\right)^{\frac{2}{3}}$$
$$h = \sqrt{a\mu\left(1-e^2\right)}$$

Since $M_e$ and $e$ are related to the eccentric anomaly and therefore true anomaly through Kepler's equation we would ideally solve it with respect to $E$. Sadly it is not possible to do so analytically and we therefore have to use numerical techniques to approximate it. Fortunately Newton's Method works well and converges fast

$$E_{n+1} = E_n - \frac{E_n - e\sin E_n - M_e}{1 - e\cos E_n}$$

The algorithm requires an initial guess for $E$ and the following works well

$$E = \begin{cases} m_e + \frac{e}{2} & \text{if } m_e < \pi \\ \\ m_e - \frac{e}{2} & \text{if } m_e > \pi \end{cases}$$

The algorithm also needs a stopping condition. Typically two conditions are provided, a max number of iterations $N$ and a tolerance $\varepsilon$. The stopping condition with $N$ is simply stop when $n == N$, if this condition is reached it likely means that the method did not converge. The other stopping condition is $|E_{n+1} - E_n| < \varepsilon|E_{n+1}|$ which likely means that the iterations have converged and a desired accuracy has been reached.

With a value for $E$ we can compute $\theta$

$$\theta = 2\arctan\left(\sqrt{\frac{1+e}{1-e}}\tan\frac{E}{2}\right)$$

## 2.8 Two-Line Elements

The Two-Line Element format is way to describe the orbital parameters for satellites in a machine readable format. An example of the format for the Hubble Space Telecope is:

```
HST
1     20580U 90037B 17257.12407589 .00000765 00000-0 36248-4 0              9997
2       20580 28.4693          139.8141    0002810 52.8630 75.3801 15.08848241303732
```

**Row 1, Field 3:** Epoch: NN|NNN.NNNNNNNN → Year|DayinYear.FractionofDay

**Row 2, Field 2:** Inclination $i$: [Degrees]

**Row 2, Field 3:** RAAN $\Omega$: [Degrees]

**Row 2, Field 4:** Eccentricity $e$: NNNNNNN $\rightarrow$ 0.NNNNNNN

**Row 2, Field 5:** Argument of perigee $\omega$: [Degrees]

**Row 2, Field 6:** Mean anomaly $M_e$: [Degrees]

**Row 2, Field 7:** NN.NNNNNNNN|NNNNN|N $\rightarrow$ Revs per day|Revs|Checksum

The mean motion is obtained from 'Revs per day' directly using

$$n = \frac{2\pi \cdot \text{Revs per day}}{24 \cdot 3600}$$

or through the orbital period

$$T = \frac{24 \cdot 3600}{\text{Revs per day}}$$
$$n = \frac{2\pi}{T}$$

For HST the Revs per day is 15.08848241. Keep in mind that the angles $M_e$, $\Omega$, $i$ and $\omega$ are given in degrees, but should be converted to radians before use. When programming it is typical to define two constants ''DTOR' $= \frac{\pi}{180}$ to convert from degrees to radians and 'RTOD' $= \frac{180}{\pi}$ to convert from radians to degrees.

## 2.9 Earth's Rotation

### 2.9.1 Time

In our everdays lives we base our timekeeping on Solar Time, i.e. the position of the sun with respect to our local position on the Earth determines the local time for us. When the sun passes over our local meridian it is said to be 12:00:00 or noon our local time. Once it has passed over our local meridian it will take around 24 hours until the next time it passes, which defines the length of our day. In the meantime we have to keep track of time ourselves by counting time intervals, which for daily activities is usually the second. If we don't have a good measure of solar time then our timekeeping won't be good either.

We can also measure how long it takes for stars to again pass over our local meridian. Because the position of the stars are essentially constant with respect to the Earth (Most stars are so far away that the relative motion is negligable) we can get very accurate measurements of how long this takes, which is 23 hours, 56 minutes and 4.095 seconds. This is just a measure how long it takes the Earth to do one full rotation and we call it Sidereal time. Since we can measure this more accurately than we can measure the Sun's apparent motion around the Earth, we use it to calculate how long it will take the Sun to again pass over our local meridian. This then becomes the basis for the length of a day and therefore the length of a time interval. This procedure of measuring Earth's rotation with respect to the fixed stars and then using a mathematical model to discern Solar Time is what we call Universal Time (UT). It is the time when Sun passes over the Greenwich Meridian offset 12 hours, such that 00:00:00 corresponds to midnight and 12:00:00 corresponds to noon. The accuracy of the

used mathematical model defines UT0 and UT1, where UT1 includes corrections for Earth's polar motion (the time difference between UT0 and UT1 is typically around 30ms). Coordinated Universal Time (UTC) which is the timekeeping system we use today and it also counts amount of hours passed since midnight. However, in contrast to UT the time interval is defined by very precise atomic clocks. This essentially means that the time interval is independent of the Earth's rotation (which does have some irregularities) and therefore over time it will drift away from UT. To correct for this drift we insert leap seconds into UTC to keep it close to UT.

**Note:** In this assignment when a time format h:min:s UTC is given, it means the time at UTC+0. Therefore if you have the local UTC time it is best to convert it to UTC+0 before using it in formulas.

### 2.9.2 Julian date

The Julian date is the number of days since January 1, 4713 B.C., 12:00. It is usually written as a decimal number of the form

DaysSinceEpoch.FractionOfDay

where FractionOfDay = 0 corresponds to 12:00:00 UT1. Julian dates are typically given with respect to UT1 and it is common to use the notation $JD_{TT}, JD_{TDT}, ...,$ for other time systems. In this course we will use $JD$ to mean $JD_{UTC}$ as an approximation, since the difference between UT1 and UTC is usually less than $\pm 0.9$s. If we are given a gregorian date (yr,mo,d) and UTC time (h,min,s) we can compute the Julian date using the equation

$$\begin{aligned} JD = 1721013.5 + 367 \cdot yr - \mathrm{INT}\left(\frac{7}{4}\left(yr + \mathrm{INT}\left(\frac{mo+9}{12}\right)\right)\right) \\ + \mathrm{INT}\left(\frac{275 \cdot mo}{9}\right) + d + \frac{1}{24}\left(h + \frac{min}{60} + \frac{s}{3600}\right) \end{aligned}$$

for instance for the date and time 2026.03.24 10:05:30

$$\begin{aligned} JD &= 1721013.5 + 367 \cdot 2026 - \mathrm{INT}\left(\frac{7}{4}\left(2026 + \mathrm{INT}\left(\frac{3+9}{12}\right)\right)\right) \\ &\quad + \mathrm{INT}\left(\frac{275 \cdot 3}{9}\right) + 24 + \frac{1}{24}\left(10 + \frac{5}{60} + \frac{30}{3600}\right) \\ JD &= 22461123.920486 \end{aligned}$$

**Note:** $\mathrm{INT}(\cdot)$ means keeping only the whole number, i.e. discarding the fractional part: $\mathrm{INT}(12.52) = 12$.

### 2.9.3 Earth's orientation with respect to ECI

In Assignment 1 we assumed that the ECEF was initially aligned with the ECI frame. Using the Julian date we can compute a close approximation to the actual sidereal angle $\theta_G$ and use it as the initial value for Earth's rotation. To

compute $\theta_G$ we start by first computing the number of Julian centuries since the J2000.0 Epoch (January 1, 2000 12:00:00.000 UTC)

$$T_0 = \frac{\text{INT}\,(JD) - 2451545}{36525}$$

With $T_0$ we can compute $\theta_{G0}$

$$\theta_{G0} = 100.4606184 + 36000.77005361 \cdot T_0 + 0.00038793 \cdot T_0^2 - 2.6 \cdot 10^{-8} T_0^3$$

which is $\theta_G$ at 00:00:00 UTC. Then we can add how much the Earth has rotated since midnight by using the fractional part of the Julian date

$$\theta_G = \theta_{G0} + \frac{180}{\pi} \cdot \omega_E \cdot (24 \cdot 3600 \cdot \text{Frac}\,(JD + 0.5))$$

Notice that we add 0.5 to the Julian date before extracting the fractional part. This is because we need the time elapsed since 00:00:00 UTC while fractional part of the Julian date gives us time elapsed since 12:00:00 UTC. Therefore we add the missing 12 hours before extracting the fractional part

$$JD = JD + 0.5$$
$$\text{UTC time} = JD - \text{INT}(JD)$$

The UTC time we get will be in fractions of a day so we convert it to seconds by multiplying with $24 \cdot 36000$. We then multiply it by the rotational speed of the Earth which is assumed to be given in 'rad/sec' which gives us how much the Earth has rotated since midnight in radians. We convert that to degrees before adding it to $\theta_{G0}$ to get $\theta_G$. The resulting $\theta_G$ could have a value outside of $[0, 2\pi]$ and if it does we need to add or subtract $360°$ until it is. A simple way to do this is (there exist many others)

$$\theta_G = \theta_G - 360 \cdot \text{TRUNC}\left(\frac{\theta_G}{360}\right)$$

generally we refer to this as angle wrapping and for angles given in radians we just substitude $2\pi$ for 360.

**Note:** The trunc function in Python's math library does not actually compute truncation as per IEEE-754. Therefore make sure you use numpy's version of trunc instead.

# 3 Extending orbit_lib.py and propagating orbits

**Assignment 2**

1. In orbit_lib.py add/update the following constants:

   $R_E$, $\omega_E$, $\mu$, DTOR, RTOD

2. Extend orbit_lib.py with the following helper functions:
   mean_anomaly_from_eccentric_anomaly($E$,$e$)
   orbital_period_from_semi_major_axis($a$)
   orbital_period_from_Revs_per_day(Revs per day)
   eccentric_anomaly_from_true_anomaly($\theta$)
   true_anomaly_from_eccentric_anomaly(E)
   orbit_params_from_tle_params($e$,Revs per day,$M_e$,$\Omega$,$i$,$\omega$)
   tle_params_from_orbit_params($h$,$e$,$\theta$,$\Omega$,i,$\omega$)
   rotation_matrix_from_classical_euler_sequence($\Omega$,i,$\omega$)
   quaternion_from_classical_euler_sequence($\Omega$,i,$\omega$)
   rotation_matrix_from_roll_pitch_yaw_sequence($\Omega$,i,$\omega$)
   quaternion_from_roll_pitch_yaw_sequence($\Omega$,i,$\omega$)
   angle_wrap_radians(angle_in_radians)
   angle_wrap_degrees(angle_in_degrees)

3. Extend the orbit_.py with the following algorithms implemented as functions:
   Algorithm 1 $\rightarrow$ sidereal_angle(JD)
   Algorithm 2 $\rightarrow$ state_from_orbit_params($h$,$e$,$\theta$,$\Omega$,i,$\omega$)
   Algorithm 3 $\rightarrow$ state_from_tle_params($e$,Revs per day,$M_e$,$\Omega$,$i$,$\omega$)
   Algorithm 4 $\rightarrow$ orbit_params_from_state($\boldsymbol{r}^i$,$\boldsymbol{v}^i$)
   Algorithm 6 $\rightarrow$ epoch_to_julian_date(epoch)
   Algorithm 7 $\rightarrow$ eccentric_anomaly_from_mean_anomaly($M_e$,$e$)

4. Find the TLE of a satellite on `https://www.celestrak.com/satcat/search.asp` and create a scenario that simulates its orbit over 1 orbital period. Use the TLE parameters to compute the initial position and velocity using state_from_TLE_params. Use Algorithm 5 to propagate the orbit.

5. Include true Earth rotation in your simulations by using the TLE Epoch together with epoch_to_julian_date(epoch) and sidereal_angle(JD) as the initial condition for the Earth propagation from Assignment 1.

**Note:** I have just given the functions some suggested names and arguments. It is not a requirement that you follow this, since there are many ways to implement these type of things in software. Since this is not a course of 'programming best practices' I will keep my suggestions as simple as possible.

# A Algorithms

## A.1 Algorithm 1: Calculating $\theta_G$ from date and time (UTC+0)

**Input:** yr,mo,d h:min:s UTC+0

**Output:** $\theta_G$

**Step 1:** Compute JD from yr,mo,d, h:min:s UTC+0.

**Step 2:** Compute Julian centuries since Epoch

$$T_0 = \frac{\mathrm{INT}\,(JD) - 2451545}{36525}$$

**Step 3:** Compute $\theta_{G0}$ from $T_0$

$$\theta_{G0} = 100.4606184 + 36000.77005361 \cdot T_0 + 0.00038793 \cdot T_0^2 - 2.6 \cdot 10^{-8} T_0^3$$

**Step 4:** Add time offset from 00:00:00 UTC

$$\theta_G = \theta_{G0} + \frac{180}{\pi} \cdot \omega_E \cdot (24 \cdot 3600 \cdot \mathrm{Frac}\,(JD + 0.5))$$

**Step 5:** Angle wrap the result such that $0° \leq \theta_G \leq 360°$.

## A.2 Algorithm 2: Satellite position and velocity in ECI frame from Classical orbital parameters

**Input:** $h$, $e$, $\theta$, $\Omega$, $i$ and $\omega$

**Output:** $\boldsymbol{r}^i$, $\boldsymbol{v}^i$

**Step 1:** Use $h, e$ and $\theta$ to calculate the satellite position and velocity in the perifocal frame

$$r = \frac{h^2}{\mu}\frac{1}{1+e\cos\theta}$$

$$\boldsymbol{r}^p = \begin{bmatrix} r\cos\theta \\ r\sin\theta \\ 0 \end{bmatrix}, \qquad \boldsymbol{v}^p = \frac{h}{\mu}\begin{bmatrix} -\sin\theta \\ e+\cos\theta \\ 0 \end{bmatrix}$$

**Step 2:** Use $i, \Omega$ and $\omega$ to calculate $\boldsymbol{r}^i$ and $\boldsymbol{v}^i$

$$\boldsymbol{R}_p^i = \begin{bmatrix} \cos\Omega & -\sin\Omega & 0 \\ \sin\Omega & \cos\Omega & 0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos i & -\sin i \\ 0 & \sin i & \cos i \end{bmatrix}\begin{bmatrix} \cos\omega & -\sin\omega & 0 \\ \sin\omega & \cos\omega & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\boldsymbol{r}^i = \boldsymbol{R}_p^i \boldsymbol{r}^p$$

$$\boldsymbol{v}^i = \boldsymbol{R}_p^i \boldsymbol{v}^p$$

## A.3 Algorithm 3: Satellite position and velocity in ECI frame from TLE parameters

**Input:** $e$, Revs per day, $M_e$, $\Omega$, $i$, $\omega$

**Output:** $\boldsymbol{r}^i$ and $\boldsymbol{v}^i$

**Step 1:** Calculate orbital period from Revs per day

$$T = \frac{24}{\text{Revs per day}} \cdot 3600$$

**Step 2:** Calculate semi-major axis

$$a = \left( \frac{\sqrt{\mu} T}{2\pi} \right)^{\frac{2}{3}}$$

**Step 3:** Solve Kepler's equation numerically for eccentric anomaly

$$E_{n+1} = E_n - \frac{E_n - e \sin E_n - M_e}{1 - e \cos E_n}$$

**Step 4:** Calculate true anomaly from eccentric anomaly

$$\theta = 2 \arctan \left( \sqrt{\frac{1+e}{1-e}} \tan \frac{E}{2} \right)$$

**Step 5:** Use Algorithm 2 to get $\boldsymbol{r}^i$ and $\boldsymbol{v}^i$

## A.4 Algorithm 4: Classical orbital parameters from position and velocity

**Input:** $\boldsymbol{r}^i$, $\boldsymbol{v}^i$

**Output:** $h$, $e$, $\theta$, $\Omega$, $i$ and $\omega$

**Step 1:** Calculate $r$, $v$ and $v_r$

$$r = \|\boldsymbol{r}^i\|, \qquad v = \|\boldsymbol{v}^i\|, \qquad v_r = \frac{\boldsymbol{r}^i \bullet \boldsymbol{v}^i}{r}$$

**Step 2:** Calculate $\boldsymbol{h}^i$, $h$ and inclination

$$\begin{aligned} \boldsymbol{h}^i &= \boldsymbol{r}^i \times \boldsymbol{v}^i \\ h &= \|\boldsymbol{h}^i\| \\ i &= \arccos\left(\frac{h_z}{h}\right) \end{aligned}$$

**Step 3:** Calculate right ascencion of ascending node ($\boldsymbol{k} = \begin{bmatrix} 0 & 0 & 1 \end{bmatrix}^\top$)

$$\begin{aligned} \boldsymbol{N}^i &= \boldsymbol{k} \times \boldsymbol{h}^i \\ N &= \|\boldsymbol{N}^i\| \\ \Omega &= \begin{cases} \arccos\left(\frac{N_x}{N}\right) & N_y \geq 0 \\ 2\pi - \arccos\left(\frac{N_x}{N}\right) & N_y < 0 \end{cases} \end{aligned}$$

**Step 4:** Calculate eccentricity

$$\begin{aligned} \boldsymbol{e}^i &= \frac{1}{\mu}\left(\left(v^2 - \frac{\mu}{r}\right)\boldsymbol{r}^i - r \cdot v_r \boldsymbol{v}^i\right) \\ e &= \|\boldsymbol{e}^i\| \end{aligned}$$

**Step 5:** Calculate argument of perigee

$$\omega = \begin{cases} \arccos\left(\frac{\boldsymbol{N}^i \bullet \boldsymbol{e}^i}{N \cdot e}\right) & e_z \geq 0 \\ 2\pi - \arccos\left(\frac{\boldsymbol{N}^i \bullet \boldsymbol{e}^i}{N \cdot e}\right) & e_z < 0 \end{cases}$$

**Step 6:** Calculate true anomaly

$$\theta = \begin{cases} \arccos\left(\frac{\boldsymbol{e}^i \bullet \boldsymbol{r}^i}{e \cdot r}\right) & v_r \geq 0 \\ 2\pi - \arccos\left(\frac{\boldsymbol{e}^i \bullet \boldsymbol{r}^i}{e \cdot r}\right) & v_r < 0 \end{cases}$$

## A.5 Algorithm 5: Orbit propagation from initial position and velocity

**Input:** $\boldsymbol{r}^i$, $\boldsymbol{v}^i$ at $t_0$

**Output:** $\boldsymbol{r}^i$ and $\boldsymbol{v}^i$ from $t_0$ to $t_e$

**Initialization:**

**Step 1:** Calculate the classical orbital parameters $h$, $e$, $\theta$, $i$, $\Omega$ and $\omega$ using Algorithm 4.

**Step 2:** Convert the classical orbital parameters to Alternative orbital parameters.

**Propagation:**

**Step 3:** For each $\Delta t$

1. Update the mean anomaly

$$M_e = M_e + n\Delta t, \quad \text{If } \; M_e \geq 2\pi \rightarrow M_e = M_e - 2\pi$$

2. Compute eccentric anomaly and true anomaly

$$E_{n+1} = E_n - \frac{E_n - e\sin E_n - M_e}{1 - e\cos E_n}, \quad \theta = 2\arctan\left(\sqrt{\frac{1+e}{1-e}}\tan\frac{E}{2}\right)$$

3. Compute new $\boldsymbol{r}^i$ and $\boldsymbol{v}^i$ using Algorithm 2.

## A.6 Algorithm 6: TLE Epoch to Julian date

**Input:** TLE Epoch: Year|DayinYear.FractionofDay

**Output:** Julian date

**Step 1:** Compute the years

$$yr = \text{INT}\left(\frac{\text{Epoch}}{1000}\right)$$

**Step 2:** Compute the number of days

$$d = \text{INT}\left(\text{Epoch} - yr \cdot 1000\right)$$

**Step 3:** Get Epoch fraction (UTC in fraction of day)

$$\text{UTC} = \text{Epoch} - (yr \cdot 1000 + d)$$

**Step 4:** Check if leap correction should be included

$$\text{leap} = \begin{cases} 1 & yr \bmod 4 = 0 \text{ and } d < 59 \\ 0 & \text{otherwise} \end{cases}$$

**Step 5:** Compute JD

$$JD = 2451544.5 + yr \cdot 365 + \text{INT}\left(\frac{yr}{4}\right) + d - \text{leap} + \text{UTC}$$

## A.7 Algorithm 7: Eccentric Anomly from Mean Anomaly and Eccentricity

**Input:** $M_e$, $e$

**Output:** $E$

**Step 1:** Compute the initial guess for $E$

$$E = \begin{cases} M_e + \frac{e}{2} & \text{If } M_e < \pi \\ M_e - \frac{e}{2} & \text{If } M_e > \pi \end{cases}$$

**Step 2:** Use Newton's Method to approximate $E$. Iterate as long as: $n < N$ and $\varepsilon \cdot |E_{n+1}| < |E_{n+1} - E_n|$.

$$E_{n+1} = E_n - \frac{E_n - e \sin E_n - M_e}{1 - e \cos E_n}$$

**Note:** Experiment with specifying different values for $N$ and $\varepsilon$ to find something that works. You can easily test how well it work by choosing radom eccentric anomalies and use Kepler's equation to get the mean anomaly. Then approximate the eccentric anomaly with this algorithm for different values of $N$ and $\varepsilon$ and compare with the true value.